IFT339 – Structures de données
Classes en C++

Manuel Lafond **Université de Sherbrooke**

Classe

Concept qui définit les attributs et comportements communs à un ensemble d’objets.

- Attributs = variables membres
- Comportements = méthodes (ou fonctions)
- Voir IFT232 pour plus de détails

Instance

Objet dont le type est une classe. Chaque instance a ses propres valeurs d’attributs.

Classes en C++

```
class MaClasse{
        int x;      //attribut

        void set_x(int val_x){      //méthode
                x = val_x;
        }

        int get_x(){
                return x;
        }
};

MaClasse c1;     //une instance
MaClasse c2;     //une autre instance
```

Classes en C++

Le point . après une instance permet d'accéder aux attributs et méthodes publiques.

```
MaClasse c1;     //une instance
MaClasse c2;     //une autre instance

c1.set_x(10);
cout<<c1.get_x()<<endl;

c2.x = 20;
cout<<c2.x<<endl;
```

Niveaux d'accès

Les variables membres et méthodes ont 3 types d'accès:

public: accessible de n'importe où

private: accessible seulement à l'intérieur de la classe

protected: accessible par les classes héritantes (IFT232)

Par défaut: private

```
class MaClasse{
private:            //ce qui suit est private
        int x;      //attribut
public:             //ce qui suit est public
        void set_x(int val_x){      //méthode
                x = val_x;
        }

        int get_x(){
                return x;
        }
};

MaClasse c1;
c1.x = 10;          //ERREUR
```

Principe d'encapsulation

L'utilisateur d'une structure de données **ne devrait jamais** avoir accès à son implémentation.

On doit pouvoir changer l'implémentation sans impact sur les utilisateurs.

Principe d'encapsulation

L'utilisateur d'une structure de données **ne devrait jamais** avoir accès à son implémentation.

On doit pouvoir changer l'implémentation sans impact sur les utilisateurs.

private: les variables membres et les méthodes de manipulation interne.

public: seulement les méthodes réalisant les tâches du TDA.

Voir exemple de Point2D

Version 1: on stocke en coordonnées x, y

Version 2: on stocke en coordonnées polaires

Rien ne change de l'extérieur.

Constructeur

Méthode spéciale appelée à la création d'une instance.

```
class MaClasse{
        MaClasse(){
                //initialisation
        }
};

MaClasse c1;        //constructeur appelé ici
```

Constructeurs

On peut définir plusieurs constructeurs, avec des arguments différents. Celui à appeler est déterminé à la création.

```
class MaClasse{
        int x;
        MaClasse(){
                x = 0;
        }
        MaClasse(int val_x){
                x = val_x;
        }
        MaClasse(int a, int b){
                x = a + b;
        }
};

MaClasse c1;        //appelle MaClasse()
MaClasse c2(10); //appelle MaClasse(int val_x)
MaClasse c3(5, 5);          //MaClasse(int a, int b)
```

Destructeur

Appelé à la destruction de l'objet (quand sa mémoire est libérée). Super **hyper-important** lorsqu'on manipulera les pointeurs et la mémoire dynamique.

```
class MaClasse{
        ~MaClasse(){
                //code de nettoyage
        }
};

int main(){
        MaClasse c1;
        return 0;
}       //destructeur appelé ici
```

Destructeur

Quand est-ce qu'un objet est détruit au juste?

Destructeur

Quand est-ce qu'un objet est détruit au juste?

Si créé de façon standard, une instance est détruite à la fin de sa portée.

- Façon standard = mémoire automatique de C++

Destructeur

Quand est-ce qu'un objet est détruit au juste?

Si créé de façon standard, une instance est détruite à la fin de sa portée.

- Façon standard = mémoire automatique de C++

Portée d'une variable: les accolades { } contenant sa déclaration.

Le destructeur est appelé à l'accolade fermante } de la portée.

Destructeur

Quand est-ce qu'un objet est détruit au juste?

Si créé de façon standard, une instance est détruite à la fin de sa portée.

- Façon standard = mémoire automatique de C++

Portée d'une variable: les accolades { } contenant sa déclaration.

Le destructeur est appelé à l'accolade fermante } de la portée.

Si créé avec la mémoire dynamique, une instance n'est JAMAIS détruite, **sauf si on le demande**. (via *delete*)

Constructeur par copie

En C++, construire en passant une instance du même type initialise avec une copie.

```
MaClasse c1;
MaClasse c2(c1);    //c2 devient une copie de c1
```

Par défaut, le constructeur par copie copie chaque variable membre avec le =

```
class MaClasse{
	int x;
	int y;
	MaClasse(const MaClasse& src){
		x = src.x;
		y = src.y;
	}
};
```

Le const indique que l'argument est en lecture seule - il ne peut pas être modifié.

Constructeur par copie

Deux façons de copier une instance.

```
MaClasse c1;
MaClasse c2(c1);    //copie par constructeur
```

ou

```
c1 = c2;            //copie par assignation
```

Assignation

On peut redéfinir le comportement de l'assignation =

```
class MaClasse{
	int x;
	int y;
	void operator=(const MaClasse& src){
		x = src.x;
		y = src.y;
	}
};
```

Éviter la duplication

```
class MaClasse{
        int x;
        int y;
         void MaClasse(const MaClasse& src){
                x = src.x;
                y = src.y;
        }
        void operator=(const MaClasse& src){
                x = src.x;
                y = src.y;
        }
};
```

Éviter la duplication

```
class MaClasse{
        int x;
        int y;
         void MaClasse(const MaClasse& src){
                (*this) = src;          //redirige vers operator=
        }
        void operator=(const MaClasse& src){
                x = src.x;
                y = src.y;
        }
};
```

Éviter la duplication

(*this) = src;

this est un pointeur sur l'instance courante.

*this retourne l'instance pointée

(*this) = src;

veut dire: mettre l'instance courante égale à src

Séparation en-tête vs implémentation

La signature des méthodes est habituellement séparée de leur implémentation, comme suit.

```
class MaClasse {
	void ma_fct(int x, int& y);
};

//... plus tard, dans le fichier ou un autre fichier
void MaClasse::ma_fct(int x, int& y) {
	y = x;
}
```

Séparation en-tête vs implémentation

La signature des méthodes est habituellement séparée de leur implémentation, comme suit.

Typiquement, en-tête dans un fichier .h (ou .hpp)

Implémentation dans un fichier .cpp

La version compilée du .cpp est compilée seulement si le fichier est modifié.

Résumé

Pour créer une classe, on doit définir:

- les variables membres (privées)
- les méthodes (publiques et privées)
- les constructeurs
- le destructeur
- le constructeur par copie
- l'affectateur operator=

Templates

```
vector<int> v;          //vector paramétré par int
vector<string> v2;  //vector paramétré par string
```

Les templates permettent de créer des classes avec des types génériques.

Ces types sont déterminés à la déclaration.

```
template <typename TYPE>    //TYPE est un nom choisi par moi
class Point {
public:
        TYPE x;
        TYPE y;

        Point(TYPE _x, TYPE _y) {
                x = _x;
                y = _y;
        }
};

Point<int> p1(5, 6);
Point<double> p2(1.1, 2.2);
```

Quand on écrit

Point<int> p1;

le compilateur génère une classe Point_int où il remplace tous les TYPE par des int

Ensuite il voit

Point<double> p2

et génère une autre classe Point_double

```
template <typename TYPE>    //TYPE est un nom choisi par moi
class Point {
public:
        TYPE x;
        TYPE y;

        Point(TYPE _x, TYPE _y) {
                x = _x;
                y = _y;
        }
};

Point<int> p1(5, 6);
Point<double> p2(1.1, 2.2);
```

```
class Point_int {
public:
        int x;
        int y;

        Point(int _x, int _y) {
                x = _x;
                y = _y;
        }
};

class Point_double {
public:
        double x;
        double y;

        Point(double _x, double _y) {
                x = _x;
                y = _y;
        }
};
```

```
template <typename T>        //T est un nom choisi par moi
class Point {
public:
        T x;
        T y;

        Point(T _x, T _y) {
                x = _x;
                y = _y;
        }
};

Point<int> p1;
Point<double> p2;
```

```
template <typename TX, typename TY>        //par moi
class Point {
public:
        TX x;
                                                                TY y;

        Point(TX _x, TY _y) {
                x = _x;
                y = _y;
        }
};

Point<int, double> p(2, 2.5);
```